RELEASE CANDIDATE / 1.1.0-rc.1

# Governed AI Workflow Platform

## KI schlägt vor.
## Regeln autorisieren.
## Worker führen aus.

Eine gezielte Engineering-Referenz für nicht lieferbare Bestellungen: begrenzte Untersuchung, unabhängige Prüfung und eine dauerhafte Ausführungsgrenze.

**167**
bestandene Prüfungen
mit klarer Abgrenzung

**1**
gebuchter Sandbox-Effekt
im Absturzversuch

**2**
Versuche,
ein stabiler Beleg

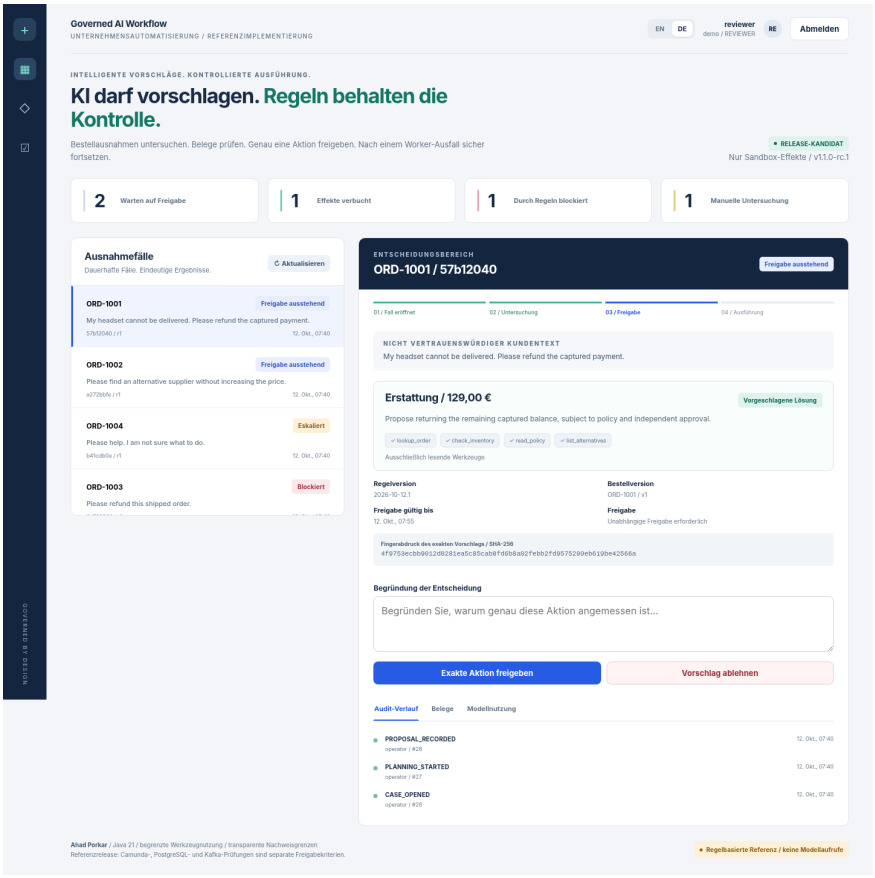
12. Oktober 2026 / Release Candidate
Deutsches Dossier. Vollständige deutsche und englische technische Leitfäden liegen dem Quellpaket bei.

Synthetische Geschäftseffekte. Externer Stack und echtes Modell sind noch abzunehmen.

01 / PRÜFOBERFLÄCHE

# Die Aktion prüfen. Nicht nur die Antwort.

Die Oberfläche macht Autorität sichtbar: Originalanfrage, genauer Vorschlag, Nachweise, Frist, unabhängige Entscheidung und Beleg.




Echte deutsche Oberfläche mit synthetischen Fällen. In Chromium mit offengelegtem DOM-Harness gerendert; kein Nachweis für native Navigation oder CSP-Durchsetzung.

| Entscheidung | Bedeutung |
|---|---|
| Sichtbarer Kontext | Kundentext bleibt nicht vertrauenswürdig. Nachweise und genaue Beträge stehen direkt bei der Entscheidung. |
| Getrennte Identitäten | Ersteller dürfen eigene Fälle nicht freigeben. Tokens werden nicht im Browserspeicher abgelegt. |

02 / AUTORITÄT UND ARCHITEKTUR

# Ein Vorschlag. Vier getrennte Verantwortungen.

Das Modell erhält kein wirksames Ausführungstool. Autorisierung, Zustand und Wiederanlauf bleiben Aufgabe der Anwendung.

01 **Untersuchen** > 02 **Validieren** > 03 **Freigeben** > 04 **Ausführen**

| Grenze | Umsetzung |
| --- | --- |
| Lesetools | Auftrag, Bestand, Regeln und zugelassene Alternativen. Leere Argumente verhindern freie Fall- oder Mandantenwahl. |
| Regeln | Striktes Schema, ganzzahlige Eurocent, begrenzte Erstattung, zulässige Stornierung und geprüfte Lieferanten. |
| Freigabe | Andere Identität; genauer Hash; Begründung und Frist. Mandant, Auftrag, Versionen und Anfrage-Hash sind gebunden. |
| Commit | Gespeicherte Freigabe und Fakten erneut prüfen. Buchung, Beleg, Audit und Outbox atomar speichern. |

Die lokale Datei erlaubt einen Schreibprozess. Der PostgreSQL-Adapter serialisiert bewusst ein JSON-Aggregat. Das macht Korrektheit nachvollziehbar, beweist aber keinen Produktionsdurchsatz.

REFUND / CANCEL / SWITCH_SUPPLIER / ESCALATE

Die ersten drei verlangen unabhängige Freigabe. Eskalation hat keine wirksamen Aktionsargumente.

03 / VERIFIKATION

# Nachweis am kritischen Punkt.

Ein echter Worker endet nach Commit und vor Bestätigung. Der Server startet neu. Ein neuer Worker wiederholt und erhält den vorhandenen Beleg.

| 106 | 29 | 9 | 23 |
|---|---|---|---|
| Java-Verträge | HTTP-/Prozesstests | DOM-Harness-Tests | Repository-Prüfungen |

| Testbereich | Nachweis |
|---|---|
| Parallelität | 32 parallele direkte und zwölf parallele HTTP-Wiederholungen werden geprüft; der Effekt bleibt einmalig. |
| Autorisierung | Mandanten/Rollen, Selbstfreigabe, Hash-Manipulation, veralteter Auftrag/Regelstand und Frist. |
| Persistenz | Rollback, Schreibsperren, Korruptionsfehler und dauerhafter Prozesswiederanlauf. |
| Modellprotokoll | Echte lokale HTTP-Fixtures prüfen den Ollama-Adapter, nicht die Qualität eines echten Modells. |

```
commit -> worker halt (73) -> server restart
retry -> same receipt -> acknowledge
1 effect / 2 attempts / refundedCents = 12900
```

Alle 167 ausgewählten Prüfungen bestanden. Neun Browser-Prüfungen nutzen einen Harness, weil native Navigation blockiert war. Maven, Spring, echte PostgreSQL-/Kafka-/Camunda-Ausführung und native Browserabnahme fehlen hier. Keine erfundene Coverage-Prozentzahl.

04 / BASELINE EVALUATION

# Baseline veröffentlichen. Schwächen ebenfalls.

48 manuell gelabelte deutsche/englische Fälle. Exakter Vorschlag bedeutet passende Aktion, Betrag und Lieferant; zulässiges Ergebnis bedeutet passenden Zustand. Beides ist kein vollständiger Geschäftserfolg.

**68,75 %**
exakte Vorschläge
33 / 48

**83,33 %**
zulässige Ergebnisse
40 / 48

| Fehlerkategorie | Erkenntnis |
|---|---|
| Teilbeträge | Eine Bitte um 25 Euro kann zum Vorschlag des gesamten verbleibenden Zahlungseingangs führen. Finanzielle Zulässigkeit beweist keine Kundenabsicht. |
| Zitate und Einschränkungen | Zitierte Aktionswörter, differenzierte Verneinung und bestimmte Lieferanten zeigen Grenzen der Schlüsselworterkennung. |
| Regelblockierung | Ein blockierter unzulässiger Vorschlag kann einen akzeptablen Zustand erreichen, ohne exakt richtig zu sein. |

Keine echten Modellaufrufe und keine ausgeführten Effekte in dieser Evaluation. Der Ollama-Adapter ist implementiert und protokollgeprüft. Ein separater Modellvergleich bleibt offen. JSON-Zeitwerte sind Beobachtungen eines synthetischen Laufs, keine SLOs.

## Evaluation reproduzieren

```
python scripts/evaluate.py
python scripts/evaluate.py --provider ollama --model MODEL
```

05 / START UND INTEGRATION

# Lokal starten. Bewusst integrieren.

Das eigenständige Baseline-Paket benötigt Java-21-JDK und Python ab 3.10. Dieser Weg braucht weder Maven noch Docker, Cloudkonto oder Modellschlüssel.

```
python scripts/run_demo.py
python scripts/worker.py --once
python scripts/test.py --repository
```

http://127.0.0.1:8095 öffnen. Erst Operator-, dann getrennten Reviewer-Token aus .local/access.json verwenden. Synthetischen Fall freigeben und Worker in einem zweiten Terminal starten. Tokens niemals veröffentlichen.

## Das Quellarchiv wird nicht stillschweigend ersetzt.

Das Archiv ist RechnungFlow, nicht das gewünschte BPMN-Repository. Es bleibt unverändert. Geliefert werden Modul und additives Overlay, kein geklonter/zusammengeführter Host. Der Installer startet als Dry-run und lehnt falsche Projekte, Symlinks, Überschreibkonflikte und V100-Kollisionen ab.

```
python scripts/apply_overlay.py TARGET --report dry-run.json
python scripts/apply_overlay.py TARGET --apply
```

Bisherige Auftragstabellen, Zahlungsanbindungen, APIs und Bestandszweige bleiben unverändert. Echte Integration braucht belegte Zahlungs-/Bestandsfakten sowie idempotente Konnektoren mit Ergebnisabgleich.

Host: Java 21 / Spring Boot 3.5.16 / Camunda 8.8.40 / PostgreSQL / Kafka. Beobachtete öffentliche Basis, kein ausgeführtes Integrationsergebnis.

06 / RELEASE-ABNAHMEN

# Ein Release Candidate. Eine glaubwürdige Engineering-Referenz.

Der Portfoliowert liegt nicht in unkontrollierter Autonomie, sondern in prüfbaren Berechtigungen, Nachweisen, Fehlerabläufen und Architekturkompromissen.

| Abnahme | Benötigter Nachweis |
| --- | --- |
| Host-Integration | Sauberen Commit dokumentieren, Overlay prüfen, Maven und PostgreSQL-Parallelität/Rollback bestehen. |
| Orchestrierung | BPMN deployen; Kafka-/Camunda-Fortschritt, doppelte Zustellung und echten Worker-Wiederanlauf nachweisen. |
| Bedienbarkeit | Native Browser-, Tastatur-/Accessibility- und benötigte Betriebssystemtests abschließen. |
| Modellqualität | Echtes Modell mit separaten Fällen evaluieren; Fehler, Zeit und Kostenannahmen offenlegen. |
| Produktionsgrenze | Demo-Identität ersetzen, alle Host-APIs sichern, Datenschutz/Lizenzen und Sicherung prüfen; echte Konnektoren bewusst einführen. |

## Primärquellen und vollständige Nachweise

[1] Ziel-Repository und beobachtete Ausgangsbasis.
[2] Natives Ollama-Tool-Protokoll.
[3] Camunda-Laufzeitlizenzen.
Eingesehen am 12. Oktober 2026. Lokale Nachweise: evidence/verification.json und benannte Test-/Evaluationsberichte. Vollständige Leitfäden: docs/en/ und docs/de/.

Anwendungscode unter MIT; Laufzeit- und Modellrechte sind separat. Keine Produktionszertifizierung, externe Veröffentlichung oder beleglose Kunden-/Arbeitgebererfolge.